BSc (Hons) in Computer Science & Artificial Intelligence

Course Work 2: Theory Report: Neural Network Principles and Critical Analysis

Project Title: SMS Spam Detection with Neural Networks

- Student Name : W.A.Thisumi Nimsani
- Student ID : KIC-BSCCSAI-251P-030

- Module : Neural Network
- Word Count: ~2,215 (Excluding equations, tables, references)
- Date of Submission: Nov 10, 2026

Table of Contents

Table of Contents **2**

Q1 Fundamentals and Numerical Backpropagation **3**

Q1(a) Foundations 3

Q1(b) Forward Pass 3

Q1(c) Backward Pass and Update 4

Q1(d) Interpretation 5

Q2 Compare Architectures and Learning Approaches **7**

Q2(a) Architecture Comparison 7

Q2(b) Learning Paradigms 7

Q2(c) Applied Judgement 8

Q3 Justify the Practical Design and Interpret Evidence **9**

Q3(a) Design Argument 9

Q3(b) Experimental Validity 10

Q3(c) Framework Limitations 10

Q4 Critically Evaluate Neural Network Developments **11**

Q4(a) Evidence-Based Comparison 11

Q4(b) Proposed Extension 12

Q4(c) Critical Synthesis 12

References **13**

Appendix **14**

Q1 Fundamentals and Numerical Backpropagation

Q1(a) Foundations

Artificial neural networks are inspired by the biological brain, though the analogy is limited. In a biological neuron the dendrites receive input from other neurons, the synapses weight that input according to the strength of the synapse and the cell only fires an action potential if a threshold is reached. The computational unit is a very direct replication of this process. You have inputs x, you have weights w, you multiply them together, you sum them, you add a bias b and you pass the result through an activation function. Two caveats on that analogy. One is that in the biological neuron the communication is in the form of discrete spikes whereas the standard network uses a continuous activation (so you could have a sigmoid or a ReLU for example) and so you lose the timing and the dynamics of the spike. The other is that the biological neuron is operating in a very complex temporal and chemical environment which a simple feedforward unit can't possibly capture. So the analogy is very useful in that it gives you a way of thinking about it, it's a very good metaphor. It's not an accurate model at all.

A single layer perceptron produces a linear decision boundary of the form $\hat{y} = sign(w^{T}x + b)$. That boundary is a straight line (or hyperplane) and so it's incapable of separating classes that are not linearly separable. The XOR problem is the classic example of this: $(0, 0)$ and $(1, 1)$ both map to 0, $(0, 1)$ and $(1, 0)$ both map to 1 and there's no single line that can separate those two groups. A multi-layer perceptron (MLP) adds one or more hidden layers with non-linear activation functions which in effect transforms the input space into a new space in which the classes may be linearly separable. The universal approximation theorem (Hornik et al., 1989) provides the formal basis for this: a network with a single sufficiently wide hidden layer is capable of approximating any continuous function. Non-linear hidden units are thus the key to increased representational capacity, turning a linear classifier into a flexible function approximator.

Q1(b) Forward Pass

Given

$x = [1, 2]^{T}$, $y = 1$, $W = [[0.2, -0.1], [0.1, 0.3]]$, $b = [0.1, -0.2]^{T}$, $v = [0.4, -0.5]^{T}$, $c = 0.2$,

and ReLU hidden activation, the forward pass proceeds as follows.

Hidden pre-activations:

$z_1 = (0.2)(1) + (-0.1)(2) + 0.1 = 0.1$

$z_2 = (0.1)(1) + (0.3)(2) - 0.2 = 0.5$

Hidden activations:

$h_1 = ReLU(0.1) = 0.1$

$h_2 = ReLU(0.5) = 0.5$

Prediction:

$\hat{y} = (0.4)(0.1) + (-0.5)(0.5) + 0.2 = -0.01$

Loss:

$L = ½(\hat{y} - y)^2 = ½(-0.01 - 1)^2 = 0.51005$

The prediction error, $\hat{y} - y = -1.01$, is a signed value that indicates the direction of the error. The loss function squares this error and divides it by two, resulting in a non-negative scalar suitable for optimization. This process removes direction but squares magnitude, penalizing larger errors more.

Q1(c) Backward Pass and Update

The output-layer gradient is straightforward:

$\partial L/\partial \hat{y} = \hat{y} - y = -1.01$

Applying the chain rule to the output weights and bias:

$\partial L/\partial v_1 = \partial L/\partial \hat{y} \cdot h_1 = -1.01 \times 0.1 = -0.101$

$\partial L/\partial v_2 = \partial L/\partial \hat{y} \cdot h_2 = -1.01 \times 0.5 = -0.505$

$\partial L/\partial c = \partial L/\partial \hat{y} \cdot 1 = -1.01$

For the hidden layer, the error signal flows back through the output weights:

$\partial L/\partial h_1 = \partial L/\partial \hat{y} \cdot v_1 = -1.01 \times 0.4 = -0.404$

$\partial L/\partial h_2 \;=\; \partial L/\partial \hat{y} \cdot v_2 \;=\; -1.01 \times (-0.5) \;=\; 0.505$

Since both pre-activations are positive, the ReLU derivative is 1 at both units:

$\partial L/\partial z_1 \;=\; -0.404,\; \partial L/\partial z_2 \;=\; 0.505$

The weight and bias gradients follow:

$\partial L/\partial W \;=\; [[-0.404,\; -0.808],\; [0.505,\; 1.010]]$

$\partial L/\partial b \;=\; [-0.404,\; 0.505]^{\mathrm{T}}$

With learning rate $\eta \;=\; 0.1$, one simultaneous gradient-descent update gives:

$W_new \;=\; [[0.2404,\; -0.0192],\; [0.0495,\; 0.199]]$

$b_new \;=\; [0.1404,\; -0.2505]^{\mathrm{T}}$

$v_new \;=\; [0.4101,\; -0.4495]^{\mathrm{T}}$

$c_new \;=\; 0.301$

Recomputing the forward pass with the updated parameters yields $z_new \;=\; [0.3424,\; 0.197]$, $h_new \;=\; [0.3424,\; 0.197]$, $\hat{y}_new \;=\; 0.35287$, and $L_new \;=\; 0.20939$. The loss has fallen from $0.51005\; to\; 0.20939$, confirming the step reduced the objective.

The error signal at a hidden unit is the product of three terms: the output error ($\partial L/\partial \hat{y}$), the output weight from the hidden unit to the output, and the derivative of the hidden activation. That product tells you how much a small change in the hidden pre-activation would impact the loss. Backprop is not a recurrent network at all. It just does one backward pass over a static feedforward computation graph. There's no hidden state carried from one time step to the next and no parameter sharing, so the network is not recurrent.

Q1(d) Interpretation

A single loss reducing step doesn't constitute convergence or generalization. Convergence is about the loss decreasing steadily over the whole training set and then plateauing. A single step on a single example tells you nothing about the rest of the data. Generalisation is about performance on data you haven't seen before. A reduction in training loss might just be the model having memorised that one example. A fairly common stopping rule is early stopping on a

validation set (i.e. stop training when the validation loss hasn't improved for a pre-defined number of epochs - the 'patience'). You could also just stop after a fixed number of epochs or after you've reached some target training loss threshold.

Q2 Compare Architectures and Learning Approaches

Q2(a) Architecture Comparison

The five architectures differ in terms of representation, connectivity, learning, supervision, application and limitations. Table 1 summarises the comparison.

Table 1. Architecture comparison

Representation	Distributed	Local (Radial)	Competitive Prototypes	Distributed + temporal	Spike trains
connectivity/state	Feedforward	Feedforward	Competitive layer	Recurrent hidden state	Recurrent event-driven
Learning	Backpropagation	Hybrid	Competitive	BPTT	STDP or surrogate gradient
Supervision	Supervised	Hybrid	Unsupervised	Supervised	Depends on rule
Application	Static classification	Function approximation	Clustering	Sequence prediction	Neuromorphic computing
Limitation	no temporal state	Centre Selction	No class labels	Vanishing gradient	Training difficulty

Temporal state only exists in Elman RNNs and SNNs. Backprop is a learning algorithm not a state. Elman RNNs use backprop through time, SNNs can use either STDP or surrogate gradients. Spiking networks don't follow one learning rule.

Q2(b) Learning Paradigms

Supervised learning (the classic example being MLP backprop) learns a mapping from inputs to labelled outputs. You need labelled training data, you work out the error between your output and the target, and you then update the weights via gradient descent. You evaluate it using accuracy, precision, recall and F1 against the ground truth labels. In the CW1 project we used labelled SMS messages to train a classifier which then predicts whether a message is ham or spam.

Unsupervised competitive learning (as in the Kohonen SOM) is all about finding structure in the data without any use of labels. The neurons compete to represent each input and the winner along with its neighbours will adjust their prototypes. Evaluation is in terms of quantisation error, topology preservation and cluster separation rather than in terms of labels. A SOM cluster identifier is not a class prediction at all. A cluster is a grouping, not a class. One cluster can contain multiple classes and one class can span multiple clusters. You have to do a post-hoc assignment of a cluster to a class using labelled data and a majority vote. That's an extra step outside of the SOM.

Q2(c) Applied Judgement

For static tabular prediction tasks the MLP is the way to go. There's no temporal dependency in the tabular data so an MLP with non-linear hidden units is perfectly capable of capturing the complex interactions. The Elman RNN is not appropriate in this case as it introduces a recurrent state that is completely unnecessary for the data. That state will be a source of vanishing gradients and adds a whole lot of unnecessary complexity.

For online sequential sensor analysis an Elman RNN is the natural choice. The sensor streams are inherently temporal and the RNN's hidden state will carry information from one time step to the next so it can react to patterns rather than just individual readings. An MLP is not the right choice in that situation as it would have no way of knowing about the temporal nature of the data (it would just process each reading independently). In the CW1 project SMS classification is a static text problem in the sense that each message is independent of the others. So both the MLP and the Text CNN are valid options. The CNN will make use of the local word order via its n-gram filters but will not have any temporal state. If the task was one of classifying sequences of messages (e.g. a chat log) then an Elman RNN would be a suitable alternative.

Q3 Justify the Practical Design and Interpret Evidence

Q3(a) Design Argument

The two CW1 neural network models (a PyTorch MLP and a Text CNN) were both trained using binary cross-entropy loss with a sigmoid output. This is the standard setup for a binary classification problem. The sigmoid output is bounded between 0 and 1 and the gradients from the binary cross-entropy loss ($dL/dz = \hat{y} - y$) are well behaved and so avoid the saturation problems that you can get with MSE when using a sigmoid output. Both models use ReLU activations in the hidden layers.

Both model depth and width were kept very modest. The MLP has two hidden layers (64 and 32 units) with 642,177 parameters. The CNN has a 32 dimensional embedding, 32 convolution filters and a linear output layer which gives a total of 234,561 parameters. With only 5,169 training messages we didn't want to go too complex with the model as there was a real risk of overfitting. The learning curves (CW1 Figures 5 and 6) show quite stable convergence with the MLP converging between epochs 9 and 11 and the CNN between epochs 17 and 23 with no signs of divergence. Both models used Xavier initialization to ensure that the signal variance was balanced across the forward and backward passes. Adam was used as the optimiser (learning rate 0.001, weight decay $1e-4$) as it has adaptive per parameter learning rates which are ideal for the sparse TF-IDF features. Dropout at 0.3 was used as a form of regularisation.

The dropout ablation (0.3 to 0.0) had a very small effect on the mean validation F1 score (a change of only −0.0018), suggesting that dropout is having very little effect at this dataset size and isn't actually harming performance. The error analysis (CW1 T5) showed the CNN had 15 test errors (7 false positives and 8 false negatives). The false positives were short ham messages that resembled spam patterns due to their brevity. This suggests that the n-gram features are not particularly effective for very short or atypical messages, whereas the MLP's TF-IDF representation may be more robust in such cases.

Q3(b) Experimental Validity

The data was split into stratified 70/15/15 splits (3,617 / 776 / 776) after the 403 duplicates had been removed. The TF-IDF vectorizer was fit on the training text only. The split indices were

saved to CSV files to keep the partitions fixed across runs. Hyperparameter tuning was restricted to one pre-specified configuration per architecture, evaluated with three random seeds and model selection was based on the validation spam F1. The test set was not accessed until the final evaluation. The validation results gave a very slight edge to the CNN, test performance gave a very slight edge to the MLP and that difference is well within the range of seed variation.

Two main threats to validity are noted. Firstly, a normalised overlap audit uncovered a very small number of case and whitespace normalised matches across the splits (train-test: 5; train-validation: 2; validation-test: 1). This would in principle allow for a very small amount of leakage of campaign templates, though the numbers are very low in comparison to the 776 test messages. Secondly, the class imbalance (13.4% spam) means that accuracy is a very poor metric to use. Spam F1 is therefore the main metric used to assess performance. The number of spam messages in the test set (98) is very small and so F1 scores will be very sensitive to any individual error.

Q3(c) Framework Limitations

PyTorch uses autograd to compute gradients, allows for GPU acceleration and has a whole range of pre-built layers. All of this reduces the amount of time you spend coding and makes it much easier to build up large models. The downside is that it can also hide problems. Autograd doesn't guarantee that the gradients are correct for your own layers. The NumPy MLP in CW1 for instance required us to do explicit gradient checks (T2c) and we would have had to do the same with any custom PyTorch layers. GPU operations can also be non-deterministic. The CW1 code sets torch.backends.cudnn.deterministic = True and benchmark = False in order to improve reproducibility, though complete determinism is not guaranteed.

Deployment constraints are also important. On a single CPU thread the CNN achieved a median latency of 0.32ms per message versus 0.94ms for the MLP. The CNN's spam F1 dropped by 0.067 under 10% character deletion whereas the MLP's F1 only dropped by 0.041. So the CNN is faster but less robust to noisy text, the MLP is slower but more robust. That trade-off is what will inform the deployment choice: the CNN is the better choice for clean text and the MLP is the better choice for noisy text.

Q4 Critically Evaluate Neural Network Developments

Q4(a) Evidence-Based Comparison

Two advances beyond the standard MLP are looked at: Mixture of Experts (MoE) as a hybrid or modular architecture and Self Supervised Learning (SSL) as a representation learning approach.

MoE involves a gating network alongside a number of expert networks. The gate selects one or a few of the experts to activate for each input, leading to conditional rather than uniform computation. Cai et al. (2025) review MoE and point out that it provides a way of increasing model capacity with very modest increases in compute. The main drawback is load balancing: some experts will be heavily used and some will be underused, and the routing can be non-differentiable which makes training difficult. For CW1 MoE is not needed at all as the dataset is small and a single CNN is sufficient. For a much larger SMS corpus with a very wide variety of spam types however a MoE of specialised CNNs could be considered.

SSL generates representations from unlabeled data via pretext tasks such as masked prediction or contrastive learning. A 2024 survey in IEEE TPAMI reports very good results in both vision and language (Gui et al., 2024), with Jing and Tian (2021) providing an earlier review of visual SSL. SSL reduces the need for labelled data but does so at the expense of requiring very large models and a lot of compute. The pretext task may also not be well aligned with the downstream task. For CW1 SSL is not feasible as we only have 5,169 messages. If we had millions of unlabeled SMS messages we could do SSL pretraining followed by fine tuning to reduce the amount of labelled data required.

Q4(b) Proposed Extension

A viable extension would be a MoE-enhanced Text CNN for multi-type spam detection. A gating network would assess each message and assign weights to three or four CNN experts (one for each of the spam types: phishing, promotional and fraud). The outputs of the experts would be combined via the gate's softmax weights and the whole thing trained end-to-end with an auxiliary load-balancing loss. The testable hypothesis is that the MoE-Text CNN will achieve at least 0.02 higher spam F1 than the single Text CNN on the subtype-annotated dataset. A fair comparison would be the CW1 Text CNN trained on the same data with the same seeds. The

primary metric would be macro-averaged spam F1 across the subtypes (with latency as a secondary metric). The experiment would use a 70/15/15 split and three seeds. The extension would be deemed unhelpful if the F1 gain is less than 0.02, if the latency more than doubles, or if the load-balancing loss doesn't prevent expert collapse.

Q4(c) Critical Synthesis

Both MoE and SSL trade off efficiency for increased capability. MoE trades off efficiency by allocating routing and capacity to achieve specialisation, SSL trades off efficiency by spending compute and data to improve label efficiency. Neither is in any way superior to a simpler model. Cai et al. (2024) and Liu et al. (2024) both have evidence in their own domain, but neither of them actually looks at SMS spam. The benefits of the proposed MoE-Text CNN are entirely hypothetical at this stage and would need to be demonstrated empirically. For CW1 the sensible approach is to go with the simpler Text CNN and save MoE for the larger, more diverse datasets where the specialisation it provides is actually going to be of benefit.

References

Cai, W., Jiang, J., Wang, F., Tang, J., Kim, S., & Huang, J. (2025). A survey on a mixture of experts in large language models. IEEE Transactions on Knowledge and Data Engineering, 37(7), 3896–3915. https://doi.org/10.1109/TKDE.2025.3554028

Fedus, W., Zoph, B., & Shazeer, N. (2022). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research, 23(120), 1–39.

jmlr.org+1

Gui, J., Chen, T., Zhang, J., Cao, Q., Sun, Z., Luo, H., & Tao, D. (2024). A survey on self-supervised learning: Algorithms, applications, and future trends. IEEE Transactions on Pattern Analysis and Machine Intelligence, 46(12), 9052–9071. https://doi.org/10.1109/TPAMI.2024.3415112

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. Neural Networks, 2(5), 359–366.

Jing, L., & Tian, Y. (2021). Self-supervised visual feature learning with deep neural networks: A survey. IEEE Transactions on Pattern Analysis and Machine Intelligence, 43(11), 4037–4058.

Appendix

GitHub Repository: The practical implementation related to this theoretical report is available at: https://github.com/ThisumiWijesinghe/SMS_SPAM_DETECTION_ANN

Google Drive: The supporting documents and relevant coursework materials are available at: https://drive.google.com/drive/folders/12Y8kfSb89Sias3Dq-CZAqhJrBI6zok1u?usp=sharing